

# Code-Layout, Readability and Reusability

## AI-Assisted Smart Budget Planner

| S.No | Code Quality Parameter    | Description                                        | Followed | Remarks                                                        |
|------|---------------------------|----------------------------------------------------|----------|----------------------------------------------------------------|
| 1    | Consistent Indentation    | Uniform Python formatting.                         | Yes      | Modular Python project.                                        |
| 2    | Proper File Structure     | Files are organized by application responsibility. | Yes      | app/main.py, auth.py, db.py, models.py, planner.py, gemini.py. |
| 3    | Meaningful Variable Names | Names reflect purpose.                             | Yes      | Planner/catalog fields are descriptive.                        |
| 4    | Function / Method Names   | Functions describe behavior.                       | Yes      | Shared planner functions and API handlers.                     |
| 5    | Code Comments / Docs      | Important behavior is documented.                  | Yes      | Project report documents architecture and behavior.            |
| 6    | Modular Design            | Logic split into reusable modules.                 | Yes      | Shared planning engine supports all three planners.            |
| 7    | No Redundant Code         | Common planning logic is reused.                   | Yes      | Home, Party and Jewelry use the shared catalog/packing engine. |
| 8    | Error Handling            | Validation and optional AI fallback are handled.   | Yes      | Image validation and Gemini fallback are documented.           |

## Reusable Components / Modules

- app/planner.py — shared catalog, item creation and budget packing.
- app/models.py — typed data structures.
- app/auth.py — authentication utilities.
- app/db.py — SQLite persistence.
- app/gemini.py — optional Gemini integration.
- app/main.py — FastAPI routes and application entry point.